

E-Panchayat Decision Support System for Gram Panchayat Administration

Honey Soni and Sujal Jagtap

School of Computing, MIT ADT University, Pune, Maharashtra, India

Abstract

Gram Panchayats require reliable tools for resident services, welfare guidance, grievance handling, development-work monitoring and meeting records. E-Panchayat is a web-based decision-support system for the 23 villages of Haveli taluka, Pune district. It combines a bilingual citizen portal with an officer dashboard, village-scoped access control, a deterministic welfare eligibility engine, grievance routing, project and Gram Sabha management, GIS visualization, audit logging and a retrieval-augmented assistant. The main design principle is that personal resident information remains inside the Panchayat application: the assistant receives only non-personal village records, while resident-specific decisions are produced locally by a rule engine. The application is deployed on Render with a FastAPI service, a React frontend and PostgreSQL persistence. Functional pilot testing was conducted for Loni Kalbhor, Theur, Wagholi and Uruli Kanchan using representative village configurations and non-personal test records. The evaluation verified role-scoped access, eligibility assessment, complaint handling, document review, meeting records, GIS rendering and bilingual assistance. The system provides a practical foundation for transparent, traceable and privacy-conscious digital administration at the Gram Panchayat level.

Keywords: *E-governance; Gram Panchayat; welfare eligibility; retrieval-augmented generation; role-based access control; rural administration.*

1 Introduction

The Panchayati Raj system makes the Gram Panchayat the closest administrative institution for many rural residents. At that level, staff must maintain resident records, communicate scheme information, register grievances, monitor development works and preserve Gram Sabha decisions. National services such as e-GramSwaraj, ServicePlus, Common Services Centres and the Digital India programme have digitized important parts of this ecosystem [1-4]. Yet an officer still needs a single operational view that links village data to day-to-day decisions.

E-Panchayat addresses this operational gap as a multi-village web application. It focuses on the questions that recur at the Panchayat counter: what scheme criteria apply, which documents are pending, where a grievance is routed, which project is relevant and what action was recorded in a Gram Sabha meeting. The system was designed for a block context rather than a single demo village, so access is constrained by both user role and village scope.

1.1 Objectives

- Digitize citizen, scheme, grievance, project, facility and Gram Sabha information in a single role-aware system.
- Evaluate welfare eligibility using transparent, data-driven rules rather than generated guesses.
- Provide English and Marathi interfaces for citizens and Panchayat staff.
- Add grounded assistance without sending individual resident records outside the application.

2 Literature Review

Existing government platforms establish the context for this work. e-GramSwaraj supports planning, accounting and progress reporting for Panchayats [1]. ServicePlus provides a configurable service-delivery framework [2], while Common Services Centres and UMANG improve assisted and mobile access to public services [3, 4]. These systems are complementary to E-Panchayat; they do not replace the need for a local interface that connects scheme rules, resident documents and operational village records.

Research on rural e-governance emphasizes adoption, local capacity and the usability of services for both citizens and officials. Kumar and Best [5] and Madon [6] show that the impact of public-sector information systems depends on institutional context, not simply on the presence of technology. Recent work on retrieval-augmented generation shows how a model can answer with retrieved evidence rather than unsupported language [7-9]. However, village-scale records contain quasi-identifiers and sensitive data, so a system must decide what is safe to retrieve and what is safe to transmit [10, 11].

Table 1. Comparison of related platforms and approaches

Platform / approach	Primary contribution	Gap addressed by E-Panchayat
e-GramSwaraj [1]	Planning, budgeting and reporting	Counter-level scheme and resident decision support
ServicePlus [2]	Configurable online services	Transparent eligibility rules and village-scoped records
CSC / UMANG [3, 4]	Assisted and mobile service access	Integrated operational dashboard and grievance workflow
Rural e-governance studies [5, 6]	Adoption and institutional context	Concrete implementation for Panchayat workflows
RAG research [7-9]	Grounded language generation	Privacy-bounded use of village data

The proposed system therefore combines the administrative coverage expected in local governance software with two technical requirements: explainable eligibility assessment and retrieval that respects the user's permissions before information is ranked or presented.

2.1 Research Gap

Many Gram Panchayat proposals list certificates, notices and complaints as isolated modules. E-Panchayat treats the connections between them as the central problem. A complaint is linked to a ward and a village; a project can be linked to the same issue; a scheme decision is linked to explicit criteria and document status. This makes a record useful for a decision, not merely available for storage.

3 Proposed System

3.1 System Overview

E-Panchayat provides two connected experiences. Citizens can review their own documents, check scheme status, submit grievances, see notices and use a bilingual helpdesk. Officers and administrators manage residents, schemes, documents, grievances, projects, facilities and Gram Sabha actions. The core service maintains a hierarchy of Maharashtra, Pune district, Haveli block and 23 villages; each village carries its Local Government Directory and Census context [12, 13].

The application separates operational decisions from language generation. Eligibility and document checks run inside the backend. The assistant retrieves only records that the current user may access, and questions that require a named resident's information are answered locally. This design supports practical assistance while preserving the confidentiality of resident files.

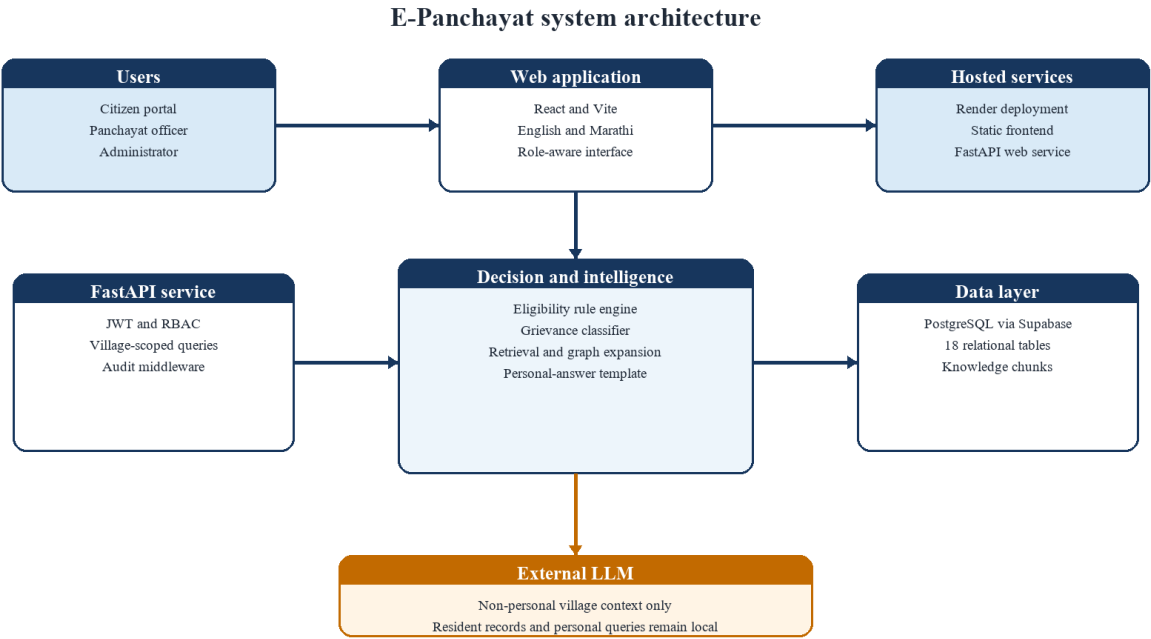


Figure 1. E-Panchayat system architecture

The deployment separates the browser application, application service and database. The React and Vite frontend is served through Render, and the FastAPI backend is deployed as a web service on Render. PostgreSQL persistence is accessed through a secured database connection, while credentials and API keys are supplied through deployment environment variables rather than embedded in the browser bundle.

3.2 Mathematical Models

Let r denote a resident record and s a welfare scheme. Each scheme holds a set of mandatory predicates C_s such as age, income, social category, ration-card type, land holding or disability threshold. Some schemes also contain alternative criteria groups $C_{\{s,j\}}$; the engine evaluates these through the any_of operator. The first expression represents the basic predicate and the second represents an alternative eligibility path.

$$P(r, s) = \bigwedge c \in C_s c(r) \quad A_s(r) = \bigvee_{j=1}^m \bigwedge c \in C_{s,j} c(r)$$

Let U indicate that a required attribute is unknown, M that the scheme requires manual review, and D that all required documents are present and verified. The system returns a four-state verdict instead of compressing incomplete records into a simple yes or no decision. This distinction is important because a missing income certificate is an action to complete, not evidence that a resident is ineligible.

$$\begin{aligned} V(r, s) &= \text{Ineligible}, P = 0 \\ \text{Needs Review}, P &= 1 \wedge (U \vee M) \\ \text{Missing Documents}, P &= 1 \wedge \neg(U \vee M) \wedge D = 0 \\ \text{Eligible}, P &= 1 \wedge \neg(U \vee M) \wedge D = 1 \end{aligned}$$

For non-personal assistant questions, K is the set of indexed village knowledge chunks and $e(\cdot)$ is an embedding function. Candidate chunks are restricted to the permissions of user u before cosine similarity is calculated. This order prevents a highly similar record from bypassing village-level access control.

$$C_u = \{k \in K \mid \text{scope}(k) \subseteq \text{perm}(u)\} \quad S(q, k) = e(q) \cdot e(k) / (||e(q)|| ||e(k)||)$$

Table 2. Core modules and design responsibilities

Module	Responsibility	Key control
Access control	Authenticates admin, officer and citizen roles	Role and village-scoped backend queries
Eligibility engine	Evaluates 29 central and state schemes	Criteria rules and document verification
Assistant	Answers village information questions	Retrieval-only personal answers
Audit trail	Records sensitive reads and state changes	HTTP middleware without request bodies

Algorithm 1. Role-scoped decision and assistant flow

Input	authenticated user u , request q , database D
1	Resolve user role and village scope from the access token.
2	If q identifies a resident, read only the permitted resident record.
3	For an eligibility request, evaluate scheme criteria and document status locally.
4	If the answer is personal, return a bilingual template with reasons; do not create an LLM request.
5	Otherwise build the candidate set from records inside the user's permitted village scope.
6	Rank retrieved knowledge chunks, expand linked records, and assemble non-personal context.
7	Generate a grounded answer from that context; fall back to a local response if the model is unavailable.
8	Return result and source labels; middleware records the auditable action.
Output	role-compliant result, sources, and audit event

4 Methodology

4.1 Development Approach

The system was developed through iterative requirements analysis, data modeling, component implementation, integration and test refinement. The requirements were derived from routine Panchayat activities: resident management, welfare support, grievance handling, development projects, Gram Sabha records, document review and public information access. The database model includes 18 relational tables covering the administrative hierarchy, users, citizens, families, schemes, documents, grievances, projects, meetings, facilities, knowledge chunks and security events.

The frontend was built as a responsive React application with English and Marathi labels. The backend uses FastAPI for REST endpoints, SQLAlchemy and Alembic for persistence and migrations, and JWT-based authentication. A rule-based bilingual classifier assigns grievance categories, priorities and departments. The map uses Leaflet and OpenStreetMap context [18], and the bilingual design follows a record-level approach so key service information can be maintained in both languages. Bhashini is a relevant national language technology initiative for future translation support [19].

4.2 Activity Flow

Figure 2 describes the activity flow used by the application. The first decision occurs when the authenticated user's role and village scope are resolved. The second decision distinguishes a personal request from a village-level question. Personal requests remain in local processing; only permitted non-personal records become context for language-model generation.

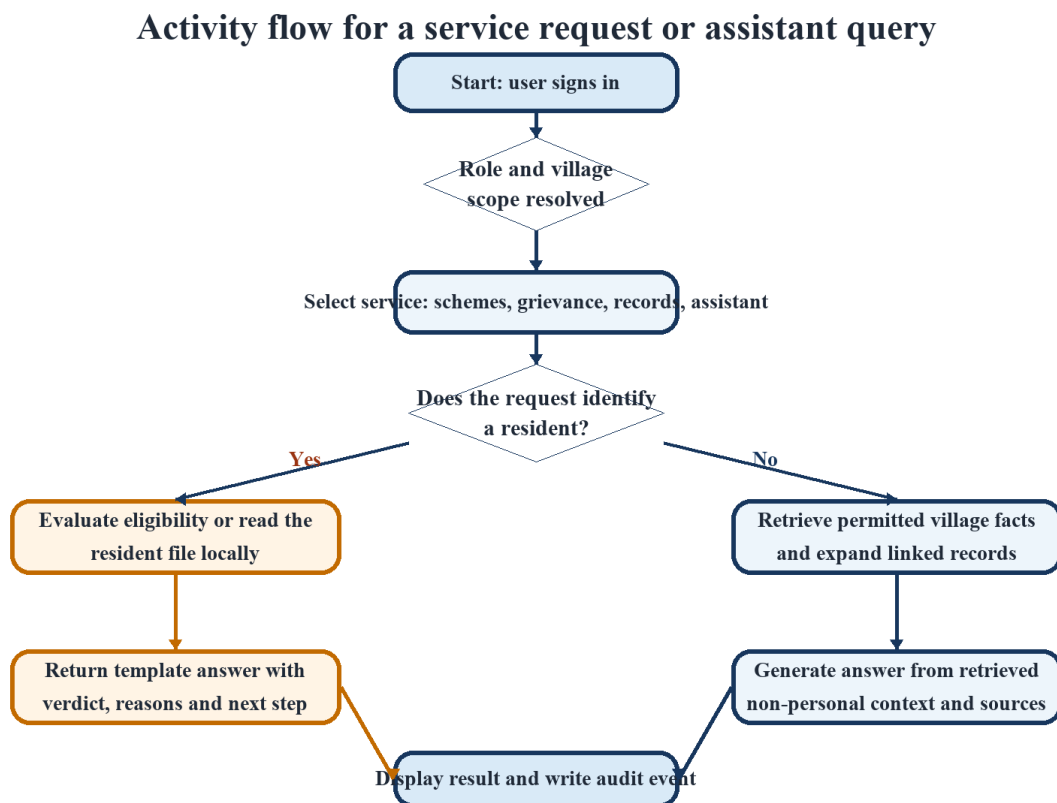


Figure 2. Activity diagram for a service request or assistant query

4.3 Pilot Testing and Deployment

Functional pilot testing was carried out using representative village configurations for Loni Kalbhor, Theur, Wagholi and Uruli Kanchan in Haveli taluka, Pune district. These locations were selected to verify that the application behaves as a multi-village system rather than as a single-village prototype. The pilot used non-personal test records and focused on completing defined administrative workflows from login to result.

The test scenarios covered role-scoped access, resident and document management, scheme eligibility assessment, grievance registration and classification, project tracking, Gram Sabha action handling, GIS display and bilingual assistance. Cross-village checks were included to confirm that an officer's view is limited to the assigned Panchayat while an administrator can view the block-level roll-up. The system was also exercised through its API test suite, which contains 260 automated tests over seeded data and security-sensitive paths.

Table 3. Functional pilot coverage and verified outcomes

Test area	Pilot activity	Verified outcome
Access and scope	Login as citizen, officer and admin; change village context	Permitted records are displayed; cross-village records are blocked
Schemes	Run criteria and document checks for 29 schemes	Verdict and reasons are returned as Eligible, Missing Documents, Needs Review or Ineligible
Grievances	Create and route bilingual complaints	Category, priority and destination department are assigned
Records and meetings	Review documents, projects and Gram Sabha actions	State changes and individual-record reads produce auditable events
Assistant and GIS	Ask village and personal queries; view village facilities	Personal queries stay local; village context is retrieved with source labels

4.4 Deployment

The completed application is deployed on Render. The deployment configuration provisions the frontend and API as separate services, exposes a health endpoint for the backend and uses environment variables for the database URL, authentication secret, model configuration and allowed browser origin. This separation keeps confidential configuration out of the client bundle and supports independent updates to the interface and backend service.

5 Results and Discussion

5.1 Functional Results

The pilot confirmed that the primary workflows operate consistently across the four selected village contexts. Citizens are limited to their own files, officers are limited to their assigned village and administrators can access the broader hierarchy. For scheme support, the result includes the decision state and explanation, enabling an officer to distinguish an ineligible resident from an otherwise eligible person who must submit or verify a document.

The implementation was tested against the same seeded dataset used by the demonstration environment. There are 10 resident records, 29 schemes, five grievances, four development projects, one Gram Sabha meeting with action items, five facilities and 23 villages in the administrative hierarchy. The data is intentionally small enough for repeatable testing, but the rules, permissions and links are the same mechanisms used by the deployed application.

Table 4. Measured system results

Measure	Result	Interpretation
Automated tests	260 passed	API, access control, audit, password reset, rate limit, registration, scheme and semantic retrieval coverage
REST endpoints	61	Services cover authentication, citizens, documents, schemes, grievances, projects, meetings, villages, analytics, audit and assistant queries
Eligibility assessments	290	10 residents evaluated against 29 schemes
Eligibility verdicts	179 Ineligible; 67 Missing Documents; 37 Needs Review; 7 Eligible	A decision is accompanied by rule- and document-level reasons
Knowledge index	67 chunks; 0 resident-derived chunks	Village knowledge is available to retrieval while resident records are not indexed
Pilot coverage	4 Haveli taluka villages	Loni Kalbhor, Theur, Wagholi and Uruli Kanchan functional configurations

5.2 Eligibility and Transparency

The eligibility engine resolves the main ambiguity found in manual scheme guidance: its answer is not merely a label. It returns the failed condition, the unknown attribute, the manual-review requirement or the missing document. For example, Sanjay Gandhi Niradhar provides alternative income and BPL paths [14], Mahatma Jyotirao Phule Jan Arogya Yojana can depend on a Yellow, Orange, AAY or Annapurna ration card [15], PMAY-G uses SECC listing conditions [16], and the Swachh Bharat Mission Gramin incentive includes alternative household conditions [17]. This level of explanation makes a decision reviewable at the Panchayat counter.

Four schemes in the current configuration use alternative any_of branches, and seven are marked for manual review because a resident record alone cannot decide their conditions. The four-state output keeps these cases visible to the officer instead of forcing a seemingly precise recommendation from incomplete evidence.

5.3 Privacy and Retrieval Behaviour

The assistant follows a retrieve-then-generate workflow for non-personal questions. It retrieves village facts, scheme information, projects, facilities, grievances and Gram Sabha records, then returns an answer with source labels. The 67-chunk index contains 29 scheme chunks, 23 village chunks, five grievance chunks, five facility chunks, four project chunks and one Gram Sabha chunk. No resident-derived chunk is present. This directly supports the design principle that the model may see village context but not an identified resident's record.

When a question concerns an individual's own record, the system uses local retrieval and a template-based explanation. The rule engine, not the language model, determines eligibility. This is important because the relevant data can include income, social category, disability information and document status. Keeping this path local avoids treating a language model as an eligibility authority and avoids transmitting personal questions for wording assistance.

5.4 Operational Discussion

The pilot also demonstrated the value of treating administration as connected workflows. A grievance can be classified into Water, Sanitation, Roads, Electricity, Health or Other, assigned a priority and routed to the appropriate department. A project record can hold budget and progress context. A Gram Sabha action item is maintained separately from the meeting summary. These links reduce the work needed to move from a citizen request to an accountable response.

Audit logging is deliberately selective. It records authenticated state-changing actions and reads of individually named records, while avoiding request bodies and passwords. Password reset is issued through a counter-based process, which is suitable for a setting where a universal SMS or email channel cannot be assumed. Together, these features make the system more practical for the office environment than a portal that only collects forms.

5.5 Pilot Observations

Across the Loni Kalbhor, Theur, Wagholi and Uruli Kanchan configurations, the same scenario scripts were completed without changing the access-control or eligibility logic. The village selection changed the records, facilities, projects and map context presented to the user; it did not change the rules that govern who may view a record. This is a useful result for block-level rollout because it validates the administrative hierarchy and per-village filtering as reusable system behaviour.

The pilot also confirmed a practical separation of responsibilities. Citizens can initiate a grievance or examine their own service information, while Panchayat staff retain the ability to review documents, update workflows and approve sensitive changes. The bilingual interface keeps core labels and explanation states available in English and Marathi. The tested flows are intentionally scenario based: they verify completion, security boundaries and traceability in the deployed application.

5.6 Limitations and Future Scope

The pilot demonstrates functional readiness across four village configurations, but larger field use should include administrative review of every scheme rule, integration with authoritative document systems and a broader user study with officials and residents. Planned extensions include OCR for uploaded records, secure DigiLocker integration, notification channels, a production vector index for larger datasets and support for additional Indian languages. These additions can extend operational reach without changing the core privacy boundary.

6 Conclusion

E-Panchayat brings together the services that Gram Panchayat staff and citizens use most often: resident records, welfare guidance, documents, grievances, projects, meetings, facilities, maps, analytics and assistance. Its key contribution is to make each decision traceable. Welfare eligibility is evaluated through criteria and documents, grievances are classified through visible rules, and sensitive actions are recorded through an audit trail.

The four-village functional pilot in Haveli taluka verified the core administrative workflows in Loni Kalbhor, Theur, Wagholi and Uruli Kanchan. The deployed Render application provides a usable platform for further rollout. By combining role-scoped access, a deterministic eligibility engine and privacy-bounded retrieval, the system offers a professional foundation for transparent and accountable Gram Panchayat digitization.

6.1 Key Contributions

- A multi-village Gram Panchayat platform for 23 Haveli taluka villages with role-based and village-scoped access.
- A 29-scheme deterministic eligibility engine with four decision states and explicit reasons.
- Privacy-bounded assistance in which resident-specific queries remain within the application.
- Functional pilot validation in four village contexts and 260 automated tests across core API workflows.

Acknowledgements

The authors thank Prof. Jyoti Gavhane for guidance and the School of Computing, MIT ADT University, for academic support. The administrative hierarchy and village data draw on public Government of India sources; all resident records used for functional testing are non-personal test data.

References

1. Ministry of Panchayati Raj, Government of India: e-GramSwaraj Portal. <https://egramswaraj.gov.in> (accessed 10 September 2026).
2. National Informatics Centre: ServicePlus Metadata-based e-Service Delivery Framework. <https://serviceonline.gov.in> (accessed 10 September 2026).
3. Ministry of Electronics and Information Technology, Government of India: Common Services Centres Scheme. <https://www.csc.gov.in> (accessed 10 September 2026).
4. Government of India: Digital India Programme. <https://www.digitalindia.gov.in> (accessed 10 September 2026).
5. Kumar, R., Best, M.L.: Impact and sustainability of e-government services in developing countries: lessons learned from Tamil Nadu, India. *The Information Society* 22(1), 1-12 (2006).
6. Madon, S.: Evaluating the developmental impact of e-governance initiatives: an exploratory framework. *Electronic Journal of Information Systems in Developing Countries* 20(1), 1-13 (2004).
7. Lewis, P. et al.: Retrieval-augmented generation for knowledge-intensive NLP tasks. In: *Advances in Neural Information Processing Systems* 33, 9459-9474 (2020).
8. Karpukhin, V. et al.: Dense passage retrieval for open-domain question answering. In: *EMNLP*, 6769-6781 (2020).
9. Edge, D. et al.: From local to global: a Graph RAG approach to query-focused summarization. *arXiv:2404.16130* (2024).
10. Sweeney, L.: k-anonymity: a model for protecting privacy. *International Journal of Uncertainty, Fuzziness and Knowledge-Based Systems* 10(5), 557-570 (2002).
11. Narayanan, A., Shmatikov, V.: Robust de-anonymization of large sparse datasets. In: *IEEE Symposium on Security and Privacy*, 111-125 (2008).
12. Ministry of Panchayati Raj, Government of India: Local Government Directory. <https://lgdirectory.gov.in> (accessed 10 September 2026).
13. Office of the Registrar General and Census Commissioner, India: Census of India 2011. <https://censusindia.gov.in> (accessed 10 September 2026).
14. Department of Social Justice and Special Assistance, Government of Maharashtra: Sanjay Gandhi Niradhar Anudan Yojana. <https://sjsa.maharashtra.gov.in> (accessed 10 September 2026).
15. Government of Maharashtra: Mahatma Jyotirao Phule Jan Arogya Yojana. <https://www.jeevandayee.gov.in> (accessed 10 September 2026).
16. Ministry of Rural Development, Government of India: Pradhan Mantri Awaas Yojana Gramin. <https://pmayg.nic.in> (accessed 10 September 2026).
17. Department of Drinking Water and Sanitation, Government of India: Swachh Bharat Mission Gramin. <https://swachhbharatmission.gov.in> (accessed 10 September 2026).
18. OpenStreetMap contributors: OpenStreetMap. <https://www.openstreetmap.org> (accessed 10 September 2026).
19. Ministry of Electronics and Information Technology, Government of India: Bhashini National Language Translation Mission. <https://bhashini.gov.in> (accessed 10 September 2026).